propsData

propsData 不是和属性有关，他用在全局扩展时进行传递数据。

类型：{ [key: string]: any }

限制：只用于 new 创建的实例中。

详细:

创建实例时传递 props。主要作用是方便测试。

首先这个使用率并不高，因为后面会有vuex,他的作用就是单页面应用中保持状态和数据的

本机就是在全局扩展里面传递参数propsData

```
<div id="app">
</div>
<template id="iswho">
        <div>
            <h3>这个是使用extend创建出来的组件</h3>
            <h4>propsData:{{news}}</h4>
        </div>
</template>

// 1.通过extend扩展一个新的组件
      var myextend=Vue.extend({
            template:'#iswho',
            // 需要通过props来接收，接收的名称得和propsData的key相同（）
            props:['news']
        })
        // 实例化对象,在里面传递数据
        new myextend({
            propsData:{
                'news':'这个就是propsData传递的数据'
        }
     }).$mount('#app')
```

ref & $refs

ref: 被用来给元素或子组件注册引用信息。引用信息将会注册在父组件的 $refs 对象上。如果在普通的 DOM 元素上使用，引用指向的就是 DOM 元素；如果用在子组件上，引用就指向组件

$refs: 一个对象，持有注册过 ref attribute 的所有 DOM 元素和组件实例。

```
<div id="app">
        <!-- 在p标签上，加上了ref特性，并赋予了id -->
        <p ref="mp">这是p标签</p>
        <button type="button" @click="get">获取ref</button>
</div>
```

```
new Vue({
    el:'#app',
    methods:{
        get(){
            // 通过this.$refs获取到的是 绑定的元素
            console.log(this.$refs.mp)
        //   var a=document.getElementById('aa')
        //   console.log(a)
        }
    }
})
```

插槽

slot翻译为插槽：

在生活中很多地方都有插槽，电脑的USB插槽，插板当中的电源插槽。

插槽的目的是让我们原来的设备具备更多的扩展性。

比如电脑的USB我们可以插入U盘、硬盘、手机、音响、键盘、鼠标等等。

组件的插槽：

组件的插槽也是为了让我们封装的组件更加具有扩展性。

让使用者可以决定组件内部的一些内容到底展示什么。

例子：左侧移动端的导航栏

导航栏我们必然会封装成一个插件，比如nav-bar组件。

一旦有了这个组件，我们就可以在多个页面中复用了。

如何封装这类组件呢？

如何去封装这类的组件呢？

它们也很多区别，但是也有很多共性。

如果，我们每一个单独去封装一个组件，显然不合适：比如每个页面都返回，这部分内容我们就要重复去封装。

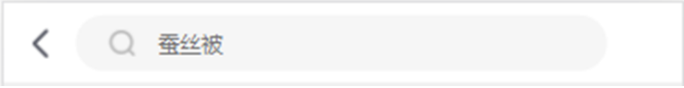
但是，如果我们封装成一个，好像也不合理：有些左侧是菜单，有些是返回，有些中间是搜索，有些是文字，等等。

如何封装合适呢？抽取共性，保留不同。

最好的封装方式就是将共性抽取到组件中，将不同暴露为插槽。

一旦我们预留了插槽，就可以让使用者根据自己的需求，决定插槽中插入什么内容。

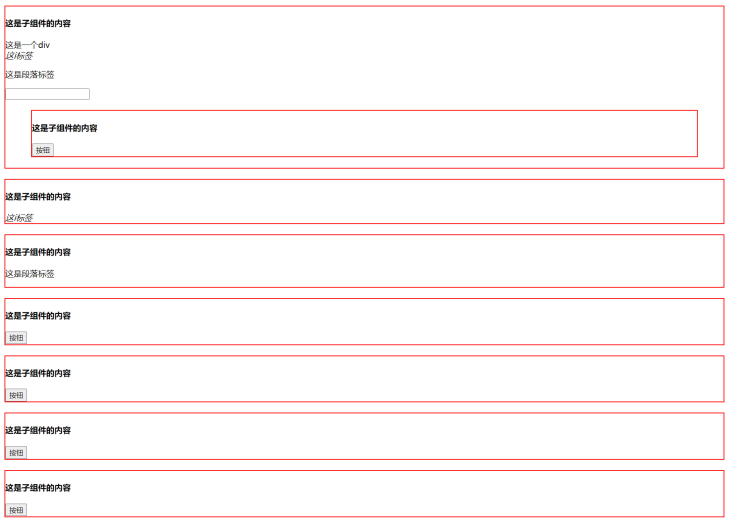
是搜索框，还是文字，还是菜单。由调用者自己来决定。



插槽基本使用

```
<!--
  1.插槽的基本使用  <slot></slot>
  2.插槽的默认值<slot><button type="button">按钮</button></slot>
  3.如果有多个值，同时放入到组件中进行替换，它会将放入的值作为元素一起替换。
-->
<div id="app">
  <!-- 使用组件 -->
  <cpn>
    <div>这是一个div</div>
    <i>这i标签</i>
    <p>这是段落标签</p>
    <input type="text" name="" id="" value="" />
    <cpn></cpn>
  </cpn>
  <cpn><i>这i标签</i></cpn>
  <cpn><p>这是段落标签</p></cpn>
  <cpn></cpn>
  <cpn></cpn>
  <cpn></cpn>
  <cpn></cpn>
</div>
<template id="mycpn">
  <div style="border: 2px solid #FF0000; margin: 20px 50px;">
    <h4>这是子组件的内容</h4>
    <!-- 使用插槽
      slot 可以理解为占位符，或者说是子组件暴露的一个让父组件传递自定义内容的接口。
      插槽的内容可以包括：HTML、其他组件。
      只要组件中的内容是可以变的，这个时候都可以考虑使用插槽
    -->
    <!-- 插槽默认值 -->
    <slot><button type="button">按钮</button></slot>
```

```
        </div>
    </template>
    // 创建子组件选项对象
        let cpn ={
            template:'#mycpn'
        }
        new Vue({
            el:'#app',
            data:{},
            components:{//注册组件
                cpn
            }
        })
```



具名插槽

过程：给slot加上name属性 <slot name="插槽名称"></slot>

友谊

```
    <div id="app">
        <!-- 使用组件 -->
        <cpn>
            <span style="color: coral;" slot="uname">友谊</span>
            <span style="color: coral;" slot="uage">22</span>
            <span style="color: coral;" slot="usex">男</span>
        </cpn>
    </div>
    <!-- 子组件的模板 -->
    <template id="mycpn">
        <div style="border: 2px solid #FF0000; margin: 20px 50px;">
            <h4>这是子组件的内容</h4>
            <!--
                具名插槽，就是给slot标签加上name属性。
             -->
            <p>姓名：<slot :name="list[0]">姓名</slot></p>
```

```
        <p>年龄：<slot name="uage">年龄</slot></p>
        <p>性别：<slot name="usex">性别</slot></p>
        {{list[0]}}
    </div>
</template>

// 创建子组件选项对象
    let cpn ={
        template:'#mycpn',
        data(){return{ list:["uname","usex","uage"]  }}
    }
    new Vue({
        el:'#app',
        data:{},
        components:{//注册组件
        cpn}
    })
```

这是子组件的内容

姓名：友谊

年龄：22

性别：男

uname

编译作用域

官方给出了一条准则：父组件模板的所有东西都会在父级作用域内编译；子组件模板的所有东西都会在子级作用域内编译。

而我们在使用my-cpn的时候，整个组件的使用过程是相当于在父组件中出现的。

那么他的作用域就是父组件，使用的属性也是属于父组件的属性。

因此，isShow使用的是Vue实例中的属性，而不是子组件的属性。

```
//了解作用域
    <!-- 父组件 -->
    <div id="app">
        <!-- 使用组件 -->
        <cpn></cpn>
        <!-- 使用的是vue实例作用于的ishow -->
        <cpn v-show="ishow"></cpn>

    </div>

    <!-- 子组件的模板 -->
    <template id="mycpn">
        <div style="border: 2px solid #FF0000; margin: 20px 50px;">
```

```
            <h4>这是子组件的内容</h4>
            <!-- 组件作用于，使用子组件的作用域 -->
            <button type="button" v-show="ishow">这是子组件中的按钮</button>
        </div>
    </template>


    // 创建子组件选项对象
    let cpn ={
     template:'#mycpn',
     data(){
        return{
          ishow:true //这个开关是控制按钮的
        }
     }
    }
     // 创建实例
     var app=new Vue({
            el:'#app',
            data:{
               ishow:true,//vue实例中的属性，这个开关是控制组件
            },
            components:{
               // 注册cpn组件
               cpn
            }
        })
```

作用域插槽

```
过程：<slot :abc="pLanguages">
                <ul v-for="item in pLanguages">
                        <li>{{item}}</li>
                </ul>
          </slot>
用法：将abc变量名，作为slot元素的一个属性绑定上去，传入值
                在组件使用时，通过v-slot="数据别名"的方式使用
<template v-slot="bb">
                        <!-- {{bb.abc}}- -->
                        {{bb.abc.join(' - ')}}
                </template>
注意：在使用作用域插槽的时候，必须使用tmeplate
```

//作用域插槽的基本使用

```
    <!-- 父组件 -->
```

```
<div id="app">
    <!-- 使用组件 -->
    <cpn></cpn>
    <!-- 以数组的形式展示 -->
    <cpn>
        <!-- 使用v-slot接收具名插槽abc的数据,并把他命名成aa(名称是自定义) -->
        <template v-slot="aa">
            {{aa.abc}}
        </template>
    </cpn>

    <!-- 以水平方向 ‘-’ 的方式来显示 -->
    <cpn>
        <!-- 使用v-slot接收具名插槽abc的数据,并把他命名成aa(名称是自定义) -->
        <template v-slot="bb">
            <!-- {{bb.abc}}- -->
            {{bb.abc.join(' - ')}}
        </template>
    </cpn>

    <cpn>
        <!-- 使用v-slot接收具名插槽abc的数据,并把他命名成aa(名称是自定义) -->
        <template v-slot="bb">
            <!-- {{bb.abc}}- -->
            {{bb.abc.join(' * ')}}
        </template>
    </cpn>
</div>

<!-- 子组件的模板 -->
<template id="mycpn">
    <div style="border: 2px solid #FF0000; margin: 20px 50px;">
        <h4>这是子组件的内容</h4>

        <!-- 作用域插槽：v-slot -->
        <!-- 定义具名插槽abc 并且传入数据pLanguages -->
        <slot :abc="pLanguages">
            <ul v-for="item in pLanguages">
                <li>{{item}}</li>
            </ul>
        </slot>
</template>

 // 创建子组件选项对象
let cpn ={
 template:'#mycpn',
 data(){
```

```
    return{
      pLanguages: ['JavaScript', 'Python', 'Swift', 'Go', 'C++']
    }
  }
}
 // 创建实例
 var app=new Vue({
        el:'#app',
        data:{
        },
        components:{
            // 注册cpn组件
            cpn
        }
    })
```